

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: Total Marks: 20

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
```

```

<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>HelloWorld
</div>
</body>
</html>

```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```

<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>

```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

I M.RESHWANTH(192411235) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M.RESHWANTH

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

CSA4305-Internet Programming

M.RESHWANTH |192411235

Case Study 1: HTTP Response Analysis

1. HTTP Response Analysis

1. Identify any incorrect or invalid fields in the header.

- **Content-Length: -150** is invalid because Content-Length cannot be negative.
- All other fields (**HTTP/1.1 200 OK**, **Date**, **Content-Type**, **Connection**) are valid.

2. Analyze the issue with Content-Length and explain its impact.

- Content-Length specifies the size of the response body in bytes.
- A negative value is invalid.
- The browser cannot determine the correct response size, causing page loading or transmission errors.

3. Determine whether the status code matches the response correctness.

- **200 OK** means the request was successful.
- Since the header contains an invalid **Content-Length**, the response is not completely correct.
- Therefore, the status code does not accurately represent the response.

4. Suggest corrections for the identified errors.

HTTP/1.1 200 OK

Date: Mon, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: 150

Connection: keep-alive

Case Study 2: DOM Structure Analysis

1. Identify missing or improperly closed elements.

- Missing `</p>` tag.
- Missing `</div>` before closing body.
- Paragraph tag is not properly closed.

2. Analyze how the browser will interpret this structure.

- Browsers automatically insert missing closing tags.
- The page may still display correctly, but the DOM structure becomes inconsistent.

3. Determine the impact on rendering and layout.

- Layout may vary between browsers.
- JavaScript DOM manipulation may not behave as expected.
- HTML validation errors occur.

4. Suggest corrections to make the DOM valid.

`<!DOCTYPE html>`

`<html>`

`<head>`

```
<title>Test Page</title>
```

```
</head>
```

```
<body>
```

```
<div>
```

```
<p>Hello World</p>
```

```
</div>
```

```
</body>
```

```
</html>
```

Case Study 3: GET vs POST in Login System

1. Interpret the differences in data transmission between GET and POST.

```
<form action="/login" method="POST">
```

```
  Username: <input type="text" name="user"><br>
```

```
  Password: <input type="password" name="pass"><br>
```

```
  <input type="submit" value="Login">
```

```
</form>
```

2. Analyze which method is more suitable for login and why.

- POST is more suitable.
- Username and password remain hidden from the URL.
- More secure than GET.

3. Identify security risks in using GET for authentication.

- Password appears in the URL.
- URLs are stored in browser history.

- URLs can be bookmarked.
- Sensitive data may be logged by servers.

4. Suggest best practices for secure login implementation.

- Use POST method.
- Use HTTPS.
- Validate user input.
- Encrypt passwords.
- Use session management and authentication tokens.

Case Study 4: HTML Code Inspection

1. Identify missing closing tags.

- Missing `</h1>`
- Missing `</p>`

2. Analyze issues with the `<img>` tag.

- Missing `alt` attribute.
- Image path may be incorrect.

3. Determine how browsers will handle these errors.

- Browsers automatically close missing tags.
- Image displays only if the file exists.
- Missing `alt` reduces accessibility.

4. Suggest a corrected version of the code.


```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>

<body>

<h1>Welcome</h1>

<p>This is a paragraph.</p>



</body>
</html>
```

Case Study 5: Browser-Server Communication Flow

1. Interpret each step in the communication flow.

1. User enters a URL in the browser.
2. DNS converts the domain name into an IP address.
3. Browser sends an HTTP request.
4. Server processes the request.
5. Server sends an HTTP response.
6. Browser renders the webpage.

2. Identify where delays can occur.

- DNS lookup.

- Slow internet connection.
- Server processing.
- Large webpage or images.

3. Analyze the role of DNS in the process.

- DNS translates the website name into an IP address.
- It allows the browser to locate the correct web server.

4. Suggest optimizations to improve performance.

- Enable browser caching.
- Use a CDN (Content Delivery Network).
- Compress images and files.
- Minify HTML, CSS, and JavaScript.
- Optimize server response time.